
Using AiM Simulator

***Intelligent System
Laboratory***

AiM Simulator setup

□ Can Use Revised Version or Original Version of AiM Sim

- AiM Simulator의 경우 제가 바꾼 부분이 거의 없습니다. 따라서 기본 버전을 사용하여도 무방합니다.

- Clone the repository

```
$ git clone https://github.com/arkhadem/aim\_simulator
```

또는

```
$ git clone https://github.com/RayKim3103/my\_aim\_simulator.git
```

- Configure the project and build the executable

```
$ mkdir build
```

```
$ cd build
```

```
$ cmake ..
```

```
$ make -j
```

```
$ cp ./ramulator2 ../ramulator2
```

```
$ cd ..
```

- Executing Simulation

```
$ ./ramulator2 -f test/example.yaml -t test/all_isr.trace > output.txt 2>&1
```

(output 리포트는 output.txt에 저장됨)

AiM Simulator setup

□ AiM Sim의 simulation 설정

- AiM Simulator의 경우 Ramulator2.0의 memory-trace parser model을 사용합니다.
 - 입력은 (R/W/AiM) (physical read/write DRAM vector Address or PIM inst) 형식으로 줌
- Configure the project and build the executable

```
$ mkdir build
$ cd build
$ cmake ..
$ make -j
$ cp ./ramulator2 ../ramulator2
$ cd ..
```
- Executing Simulation

```
$ ./ramulator2 -f test/example.yaml -t test/all_isr.trace > output.txt 2>&1
```

AiM Simulator CMDs

❑ Deep Learning CMDs

- Simulator의 CMD와 JSSCC의 CMD는 구성이 약간 다르다
- Simulator의 CMDs

TABLE I NEWLY DEFINED DEDICATED DL CMD SET		
Type	CMD	Description
Bank Activation	ACT4, ACT16	Activate four/sixteen banks in parallel
	ACTAF4, ACTAF16	Activate rows storing activation function LUTs in four/sixteen banks parallel
Compute	MACSB, MAC4B, MACAB	Perform MAC in one/four/sixteen banks in parallel
	AF	Compute activation function in all banks
	EWMUL	Perform element-wise multiplication
Data	RDCP	Copy data from a bank to global buffer
	WRCP	Copy data from global buffer to a bank
	WRGB	Write to global buffer
	RDMAC	Read from MAC result register
	RDAF	Read from activation function result register
	WRBIAS	Write bias to MAC result register
	WRBK	Write to all activated banks in parallel

[GDDR6 based AiM paper의 Table]

유형	명령어	세부 사항
AiM	WR_SBK [GPR_0] [channel_mask] [bank] [row]	GPR에서 256b를 Single Bank로 Write
AiM	WR_ABK [GPR_0] [channel_mask] [row]	GPR에서 256b를 16개 Bank로 Write
AiM	WR_GB [opsize] [GPR_0] [channel_mask]	GPR에서 256b를 GB로 Write
AiM	WR_BIAS [GPR_0] [channel_mask]	GPR에서 256b를 16개 Bank의 16b MAC Reg로 Write
AiM	RD_MAC/RD_AF [GPR_0] [channel_mask]	16개 Bank의 16b MAC/AF Reg에서 256b를 GPR로 Read
AiM	RD_SBK [GPR_0] [channel_mask] [bank] [row]	Single Bank에서 256b를 GPR로 Write
AiM	COPY_BKGB [opsize] [channel_mask] [bank] [row]	16개 Bank에서 256b를 GB로 Copy
AiM	COPY_GBBK [opsize] [channel_mask] [bank] [row]	GB에서 256b를 16개 Bank로 Copy
AiM	MAC_SBK [opsize] [channel_mask] [bank] [row]	Single Bank에서 256b MAC 수행
AiM	MAC_ABK [opsize] [channel_mask] [row]	모든 Bank에서 256b MAC 수행

AiM Simulator CMDs

❑ Deep Learning CMDs

- Simulator의 CMD와 JSSCC의 CMD는 구성이 약간 다르다
- Simulator의 CMDs

TABLE I NEWLY DEFINED DEDICATED DL CMD SET		
Type	CMD	Description
Bank Activation	ACT4, ACT16	Activate four/sixteen banks in parallel
	ACTAF4, ACTAF16	Activate rows storing activation function LUTs in four/sixteen banks parallel
Compute	MACSB, MAC4B, MACAB	Perform MAC in one/four/sixteen banks in parallel
	AF	Compute activation function in all banks
	EWMUL	Perform element-wise multiplication
Data	RDCP	Copy data from a bank to global buffer
	WRCP	Copy data from global buffer to a bank
	WRGB	Write to global buffer
	RDMAC	Read from MAC result register
	RDAF	Read from activation function result register
	WRBIAS	Write bias to MAC result register
	WRBK	Write to all activated banks in parallel

[GDDR6 based AiM paper의 Table]

유형	명령어	세부 사항
AiM	AF [channel_mask]	모든 Bank에서 Activation Func 수행
AiM	EWMUL [opsize] [channel_mask] [row]	4개 Bank에서 Elementwise Mult 수행
AiM	EWADD [opsize] [GPR_0] [GPR_1]	GPR을 사용한 Elementwise Add 수행
AiM	SYNC	ISR 명령어 간의 Barrier
AiM	EOC	계산 종료, 시뮬레이션 종료
W	MEM [channel_id] [bank] [row]	Bank에 대한 기존 Write 접근
R	MEM [channel_id] [bank] [row]	Bank로부터의 기존 Read 접근
W	GPR [GPR_0]	GPR에 256b Write
R	GPR [GPR_0]	GPR로부터 256b Read
W	CFR [CFR_0] [data]	Data를 이용해 CFR 구성

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
1. # AiM [opcode] [opsize] [GPR_id_0] [GPR_id_1] [channel_mask] [bank_id] [row_id]
2. # R/W GPR [GPR_id]
3. # R/W CFR "[CFR_id {0: broadcast 1: EWUL_bg 2: afm}]" [data]
4. # R/W MEM [channel_id] [bank_id] [row_id]
5. # Access granularity [GPR: BURST bytes] [CFR: 2 bytes] [MEM: Burst bytes] where BURST = 32 bytes
6.
7. # Regular write to row [8-11] in bank [2] of channels [0-3]
8. # R/W MEM [channel_id] [bank_id] [row_id]
9. W MEM 0 2 8
10. W MEM 1 2 9
11. W MEM 2 2 10
12. W MEM 3 2 11
13.
14. # Writing to GPR [0-1] for bank data
15. # R/W GPR [GPR_id]
16. W GPR 0
17. W GPR 1
18.
19. # Writing to GPR [2-3] for global buffer data
20. # R/W GPR [GPR_id]
21. W GPR 2
22. W GPR 3
23.
24. # Writing to GPR [4-7] for bias data
25. # R/W GPR [GPR_id]
26. W GPR 4
27. W GPR 5
28. W GPR 6
29. W GPR 7
30.
31. # "Write [2-data = 512 bits] from GPR[0 1] to channels [0-3] bank [0] row [0] (channel mask must only specify 1 channel)"
32. # AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]
33. AiM WR_SBK 0 1 0 0
34. AiM WR_SBK 0 2 0 0
35. AiM WR_SBK 0 4 0 0
36. AiM WR_SBK 0 8 0 0
37. AiM WR_SBK 1 1 0 0
38. AiM WR_SBK 1 2 0 0
39. AiM WR_SBK 1 4 0 0
40. AiM WR_SBK 1 8 0 0
41.
42. # "Write [2-data = 512 bits] from GPR[2 3] to channels [0-3] global buffers"
43. # AiM [opcode] [opsize] [GPR_id_0] [channel_mask]
44. AiM WR_GB 2 2 15
45.
46. # Write [4-data = 1024 bits] from GPR[4-7] to channels [0-3] * 16 MAC accumulators
47. # AiM [opcode] [GPR_id_0] [channel_mask]
48. AiM WR_BIAS 4 15
49.
```

- Simul이기에, Data는 따로 주고 받지 않고, Address만 이용합니다.

1. **Memory Write (line 9-12)** - R/W MEM [channel_id] [bank_id] [row_id]

- W MEM 0 2 8 : Regular write to row[8] in bank[2] of channel[0]

2. **GPR Write (line 16-29)** - R/W GPR [GPR_id]

- W GPR 0 : Writing 256bit to GPR[0] (GPR: General Purpose Register)

3. **BANK Write (line 33-40)** - AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]

- AiM WR_SBK 0 1 0 0 : Write 256bit from GPR[0] to channel[0] bank[0] row[0]

4. **GB Write (line 44)** - AiM [opcode] [opsize] [GPR_id_0] [channel_mask]

- AiM WR_GB 2 2 15 :

Write [2-data = 512 bits] from GPR[2-3] to channels [0-3] global buffers

- opsize는 가져올 data 크기를 조절
- opsize = 2이면, 256b data 2개 (= 512b) 가져옴
- GPR은 1개 당 256b 저장 가능
- 512b 가져오려면 GPR id: 2부터 시작해 GPR id: 3까지 이용해야 함
- channel mask: 15 = 32'b0000_1111이므로 channel 0~3을 의미

4. **16개 BANK의 16b MAC Reg에 Write (line 48)** - AiM [opcode] [GPR_id_0] [channel_mask]

- AiM WR_BIAS 4 15 :

Write [4-data = 1024 bits] from GPR[4-7] to channels [0-3] * 16 MAC accumulators

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
50. # set CFR[mr13.broadcast at address 0x0010] to 1 (MAC from global buffer)
51. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
52. W CFR 0 1
53.
54. # MAC operation between 2 columns of all banks in channels [0-3] with 2 columns of global buffer
55. # AiM [opcode] [opsize] [channel_mask] [row_id]
56. AiM MAC_ABK 2 15 0
57.
58. # set CFR[mr13.broadcast at address 0x0010] to 0 (MAC from next bank)
59. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
60. W CFR 0 0
61.
62. # MAC operation between 2 columns of half of the banks in channels [0-3] with 2 columns of the next banks
63. # AiM [opcode] [opsize] [channel_mask] [row_id]
64. AiM MAC_ABK 2 15 0
65.
66. # set CFR[mr13.broadcast at address 0x0010] to 1 (MAC from global buffer)
67. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
68. W CFR 0 1
69.
70. # MAC operation between 2 columns of bank [0] in channels [0-3] with 2 columns of global buffer
71. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
72. AiM MAC_SBK 2 15 0 0
73.
74. # set CFR[mr13.broadcast at address 0x0010] to 0 (MAC from next bank)
75. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
76. W CFR 0 0
77.
78. # MAC operation between 2 columns of bank [0] in channels [0-3] with 2 columns of the next banks
79. # AiM [opcode] [opsize] [channel_mask] [bank_id] [row_id]
80. AiM MAC_SBK 2 15 0 0
81.
82. # set CFR[mr14.ewmul_bg at address 0x0018] to 1 (EWMUL only 1 bankgroup)
83. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
84. W CFR 1 1
85.
86. # "ELMUL operation between 1 bank group 2 columns of channels [0-3] with 2 columns of the next banks"
87. # AiM [opcode] [opsize] [channel_mask] [row_id]
88. AiM EWMUL 2 15 0
89.
90. # set CFR[mr14.ewmul_bg at address 0x0018] to 0 (EWMUL in all bankgroups)
91. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
92. W CFR 1 0
93.
94. # "ELMUL operation between all bank groups 2 columns of channels [0-3] with 2 columns of the next banks"
95. # AiM [opcode] [opsize] [channel_mask] [row_id]
96. AiM EWMUL 2 15 0
97.
98. # set CFR[mr13.afm at address 0x001C] to 1 (AF number 1)
99. # R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]
100. W CFR 2 1
```

5. **set CFR (line 52)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 0 1 : set CFR[0] to 1 (= MAC from global buffer)

6. **MAC Operation (line 56)** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM MAC_ABK 2 15 0 :

MAC : (2 columns of all banks in channels [0-3]) x (2 columns of global buffer)

7. **set CFR (line 60)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 0 0 : set CFR[0] to 0 (= MAC from next bank)

8. **MAC Operation (line 64)** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM MAC_ABK 2 15 0 :

MAC : (2 columns of half banks in channels [0-3]) x (2 columns of the next banks)

... (생략) ...

9. **set CFR (line 84)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 1 1 : set CFR[1] to 1 (= EWMUL only 1 bankgroup)

10. **EWMUL Operation (line 88)** - AiM [opcode] [opsize] [channel_mask] [row_id]

- AiM EWMUL 2 15 0 :

(2 columns of 1 bank group in channels [0-3]) x (2 columns of next banks)

같은 bankgroup 내 -> Bank[i] × Bank[i+1] (simulation상에서 bankgroup id는 구현 X)

... (생략) ...

11. **set CFR (line 100)** - R/W CFR "[CFR_id (0: broadcast 1: EWUL_bg 2: afm)]" [data]

- W CFR 2 1 : set CFR[2] to 1 (= Activation Function 1, simul 상에서 Act Func들 정의는 X)

AiM Simulator Example Trace

□ 기본으로 주어진 all_isr.trace를 분석

```
101.
102. # AF operation in all banks of channels [0-3]
103. # AiM [opcode] [channel_mask]
104. AiM AF 15
105.
106. # Reading MAC accumulators of all banks of channels [0-3] to GPRs[8-11]
107. # AiM [opcode] [GPR_id_0] [channel_mask]
108. AiM RD_MAC 8 15
109.
110. # Reading AF results of all banks of channels [0-3] to GPRs[12-15]
111. # AiM [opcode] [GPR_id_0] [channel_mask]
112. AiM RD_AF 12 15
113.
114. # ELADD operation between GPRs[8-11] and GPRs[12-15]
115. # AiM [opcode] [opcode] [GPR_id_0] [GPR_id_1]
116. AiM EWADD 4 8 12
117.
118. # Copying 2 columns from bank 0 row 0 to global buffers in channels [0-3]
119. # AiM [opcode] [opcode] [channel_mask] [bank_id] [row_id]
120. AiM COPY_BKGB 2 15 0 0
121.
122. # Copying 2 columns from global buffers to bank 0 row 0 in channels [0-3]
123. # AiM [opcode] [opcode] [channel_mask] [bank_id] [row_id]
124. AiM COPY_GBBK 2 15 0 0
125.
126. # Read [2-data = 512 bits] from channels [0-3] bank [0] row [0] to GPR[20-27] (channel mask must only specify 1 channel)
127. # AiM [opcode] [GPR_id_0] [channel_mask] [bank_id] [row_id]
128. AiM RD_SBK 20 1 0 0
129. AiM RD_SBK 21 1 0 0
130. AiM RD_SBK 22 2 0 0
131. AiM RD_SBK 23 2 0 0
132. AiM RD_SBK 24 4 0 0
133. AiM RD_SBK 25 4 0 0
134. AiM RD_SBK 26 8 0 0
135. AiM RD_SBK 27 8 0 0
136.
137. # AiM [opcode] [GPR_id_0] [channel_mask] [row_id]
138. AiM WR_ABK 4 1 5
139. AiM WR_ABK 5 2 5
140. AiM WR_ABK 6 4 5
141. AiM WR_ABK 7 8 5
142.
143. # Finishing compute (MUST have this instruction at the end of the trace file)
144. # AiM [opcode]
145. AiM EOC
```

12. **AF Operation (line 104)** - AiM [opcode] [channel_mask]

- AiM AF 15:

모든 bank의 MAC accumulator 값에 AF를 적용 & AF regs에 저장

13. **Read MAC regs (line 108)** - AiM [opcode] [GPR_id_0] [channel_mask]

- AiM RD_MAC :

Read MAC accumulators of all banks in channel[0-3] to GPRs[8-11]

... (생략) ...

AiM Simulator Example Trace Output

❑ Simulation Output Report

Frontend: impl: AiMTrace	Controller: impl: AiM id: Channel 0 CH0_precharged_cycles: 0 # total number of precharged cycles CH0_num_UNKNOWN_commands: 0 # total number of UNKNOWN commands CH0_num_TMOD_commands: 0 # total number of TMOD commands CH0_num_WRMAC16_commands: 0 # total number of WRMAC16 commands CH0_num_RDAF16_commands: 0 # total number of RDAF16 commands CH0_num_WRGB_commands: 0 # total number of WRGB commands CH0_num_WRCPC_commands: 0 # total number of WRCPC commands CH0_num_EWMUL16_commands: 0 # total number of EWMUL16 commands CH0_num_AF16_commands: 0 # total number of AF16 commands CH0_num_MAC16_commands: 0 # total number of MAC16 commands CH0_num_MAC_commands: 0 # total number of MAC commands CH0_num_PRE4_commands: 0 # total number of PRE4 commands CH0_num_ACT16_commands: 0 # total number of ACT16 commands CH0_idle_cycles: 679 # total number of idle cycles CH0_AiM_ISR_SYNC_cycles: 0 # total number of AiM_ISR_SYNC cycles CH0_num_WR_commands: 1 # total number of WR commands CH0_num_RDMAC16_commands: 0 # total number of RDMAC16 commands CH0_AiM_ISR_WR_ABK_cycles: 0 # total number of AiM_ISR_WR_ABK cycles CH0_AiM_ISR_RD_SBK_cycles: 0 # total number of AiM_ISR_RD_SBK cycles CH0_AiM_ISR_COPY_GBBK_cycles: 0 # total number of AiM_ISR_COPY_GBBK cycles CH0_num_ACT4_commands: 0 # total number of ACT4 commands CH0_active_cycles: 689 # total number of active cycles CH0_num_EOC_commands: 0 # total number of EOC commands CH0_AiM_ISR_RD_AF_cycles: 0 # total number of AiM_ISR_RD_AF cycles CH0_AiM_ISR_WR_AFLUT_cycles: 0 # total number of AiM_ISR_WR_AFLUT cycles CH0_num_RDPC_commands: 0 # total number of RDPC commands CH0_AiM_ISR_COPY_BKGB_cycles: 0 # total number of AiM_ISR_COPY_BKGB cycles CH0_num_PRE_commands: 0 # total number of PRE commands CH0_AiM_ISR_WR_BIAS_cycles: 0 # total number of AiM_ISR_WR_BIAS cycles CH0_AiM_ISR_WR_GB_cycles: 0 # total number of AiM_ISR_WR_GB cycles CH0_num_REFab_commands: 0 # total number of REFab commands CH0_W_cycles: 29 CH0_num_SYNC_commands: 0 # total number of SYNC commands CH0_AiM_ISR_MAC_SBK_cycles: 0 # total number of AiM_ISR_MAC_SBK cycles CH0_num_RDA_commands: 0 # total number of RDA commands CH0_AiM_ISR_AF_cycles: 0 # total number of AiM_ISR_AF cycles CH0_num_ACT_commands: 1 # total number of ACT commands CH0_AiM_ISR_EWMUL_cycles: 0 # total number of AiM_ISR_EWMUL cycles CH0_num_WRA16_commands: 0 # total number of WRA16 commands CH0_R_cycles: 0 CH0_AiM_ISR_EWADD_cycles: 0 # total number of AiM_ISR_EWADD cycles CH0_AiM_ISR_RD_MAC_cycles: 0 # total number of AiM_ISR_RD_MAC cycles CH0_AiM_ISR_EOC_cycles: 0 # total number of AiM_ISR_EOC cycles CH0_num_PREA_commands: 0 # total number of PREA commands CH0_not_issued_cycles: 7 # total number of not issued cycles CH0_num_RD_commands: 0 # total number of RD commands CH0_AiM_ISR_MAC_ABK_cycles: 0 # total number of AiM_ISR_MAC_ABK cycles CH0_num_WRA_commands: 0 # total number of WRA commands CH0_AiM_ISR_WR_SBK_cycles: 0 # total number of AiM_ISR_WR_SBK cycles CH0_num_REFpb_commands: 0 # total number of REFpb commands Scheduler: impl: FRFCFS RefreshManager: impl: A11Bank
MemorySystem: impl: AiMDRAM total_num_AiM_ISR_SYNC_requests: 0 # total number of AiM_ISR_SYNC requests total_num_AiM_ISR_WR_ABK_requests: 4 # total number of AiM_ISR_WR_ABK requests total_num_AiM_ISR_EWADD_requests: 1 # total number of AiM_ISR_EWADD requests total_num_AiM_ISR_AF_requests: 1 # total number of AiM_ISR_AF requests total_num_AiM_ISR_MAC_ABK_requests: 2 # total number of AiM_ISR_MAC_ABK requests total_num_AiM_ISR_MAC_SBK_requests: 2 # total number of AiM_ISR_MAC_SBK requests total_num_AiM_ISR_COPY_BKGB_requests: 1 # total number of AiM_ISR_COPY_BKGB requests total_num_AiM_ISR_RD_AF_requests: 1 # total number of AiM_ISR_RD_AF requests total_num_AiM_ISR_RD_SBK_requests: 8 # total number of AiM_ISR_RD_SBK requests total_num_AiM_ISR_RD_MAC_requests: 1 # total number of AiM_ISR_RD_MAC requests total_num_AiM_ISR_EOC_requests: 1 # total number of AiM_ISR_EOC requests memory_system_cycles: 689 total_num_AiM_ISR_EWMUL_requests: 2 # total number of AiM_ISR_EWMUL requests total_num_R_MEM_requests: 0 # total number of R MEM requests total_num_wait_read_stalls: 0 # total number of cycles that AiM DMA is stalled because of waiting for read operations from channels total_num_ISR_full: 0 # total number of cycles that AiM DMA does not receive ISR because of lack of enough ISR space total_num_W_CFR_requests: 7 # total number of W CFR requests total_num_R_CFR_requests: 0 # total number of R CFR requests total_num_AiM_ISR_WR_GB_requests: 1 # total number of AiM_ISR_WR_GB requests total_num_W_GPR_requests: 8 # total number of W GPR requests total_num_AiM_ISR_WR_AFLUT_requests: 0 # total number of AiM_ISR_WR_AFLUT requests total_num_AiM_ISR_COPY_GBBK_requests: 1 # total number of AiM_ISR_COPY_GBBK requests total_num_W_MEM_requests: 4 # total number of W MEM requests total_num_AiM_ISR_WR_SBK_requests: 8 # total number of AiM_ISR_WR_SBK requests total_num_R_GPR_requests: 0 # total number of R GPR requests total_num_AiM_ISR_WR_BIAS_requests: 1 # total number of AiM_ISR_WR_BIAS requests DRAM: impl: GDDR6 AddrMapper: impl: RoBaRaCoCh	Channel [31] 까지 Report 제공 ● ● ●

Channel [31] 까지 Report 제공

• • •

- Memory System
 - 각각의 Request가 들어온 횟수
 - request_queue가 꽉찬 cycles
 - AiM DMA Blocking에 의해 stall 된 cycles
- Memory Contorller
 - 각각의 Command가 들어온 횟수
 - Command가 소비한 cycles
- DRAM power를 AiM simulator안에 구현 안 하였기에 power report는 제공 X